P.PORTO

ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO

Relatório

Processamento Estruturado de Informação

2025/2026

Relatório elaborado por:

Grupo 10

8240344 Mariana Silva Barreto

8240268 David Gabriel Ventura Fernandes

8240227 Ricardo Filipe Fernandes Lopes

Curso de:

Licenciatura em Engenharia Informática

Docentes:

Bruno Oliveira (bmo@estg.ipp.pt)

Mariana Carvalho (mrc@estg.ipp.pt)

Pedro Pinto (plp@estg.ipp.pt)

Conteúdo

1. Introdução..... 5

2. Contextualização e Caracterização do Caso de Estudo..... 7

3. Implementação..... 9

4. Apreciação Crítica..... 23

5. Conclusão..... 24

6. Referências..... 25

Introdução

O presente relatório detalha o desenvolvimento de um sistema de monitorização hospitalar para a entidade **Health Time**, cujo propósito central é a uniformização e o acompanhamento rigoroso dos tempos de espera nos serviços de urgência, consulta e cirurgia. Face à complexidade inerente à gestão de dados de saúde provenientes de múltiplas unidades parceiras, o projeto foca-se na criação de um sistema digital capaz de processar fluxos de informação periódicos de forma eficiente e consistente. O objetivo deste trabalho consiste no desenvolvimento de uma solução tecnológica capaz de processar estes dados e fornecer análises críticas em tempo real através de uma REST API e de visualizações dinâmicas, facilitando a monitorização e a elaboração de relatórios comparativos entre as diferentes instituições de saúde.

No que se refere à contextualização, a abordagem adotada baseia-se na utilização de tecnologias NoSQL, especificamente o MongoDB, como pilar central para o armazenamento e integração de grandes volumes de dados. Esta escolha justifica-se pela necessidade crítica de lidar com dados, que incluem tanto novos documentos submetidos em formato XML pelos hospitais como o histórico de dados proveniente de ficheiros CSV disponibilizados.

A estrutura do sistema assegura que toda a informação recebida seja validada e transformada através de pipelines de integração, utilizando a Aggregation Framework do MongoDB para organizar a informação de acordo com as melhores práticas de modelação.

Posteriormente, estes dados são disponibilizados em **formato JSON**, servindo de base para o cumprimento dos requisitos analíticos e para a visualização intuitiva dos resultados através de dashboards no MongoDB Atlas.

Este trabalho pretende, assim, demonstrar a aplicabilidade de vocabulários estruturados e bases de dados flexíveis num domínio de negócio que exige respostas rápidas, clareza e capacidade de expansão futura.

Contextualização e Caracterização do Caso de Estudo

2.1. Descrição do Domínio de Negócio

O domínio de negócio deste projeto centra-se na monitorização e gestão da eficiência hospitalar, especialmente no que diz respeito aos tempos de resposta nos serviços de saúde.

A Health Time funciona como uma entidade central de informação, cuja missão principal é monitorizar de forma uniforme os tempos de espera hospitalares, abrangendo os serviços de urgência, consulta e cirurgia. A sua relevância reside na capacidade de converter dados isolados, provenientes de diversas instituições parceiras, em indicadores de desempenho transversais que permitem realizar análises comparativas e relatórios detalhados.

Exemplos práticos deste domínio incluem a gestão de fluxos de triagem em serviços de urgência, onde a monitorização da rapidez na observação médica inicial é um fator crítico para a segurança do paciente. Da mesma forma, na administração de listas de espera para cirurgias programadas, a distinção entre listas de espera gerais e oncológicas é determinante para garantir a qualidade e a priorização do cuidado prestado. Para assegurar a consistência, clareza e a futura expansão do sistema, a integridade desta informação é garantida através de um vocabulário XML comum. Este padrão permite que diferentes sistemas hospitalares comuniquem de forma uniforme com a plataforma central, facilitando o armazenamento e a integração dos dados numa base de dados MongoDB.

2.2. Análise do Problema

O principal desafio identificado neste domínio é a fragmentação e heterogeneidade da informação hospitalar, exigindo um esforço de uniformização por parte da Health Time. As necessidades específicas incluem a recolha periódica de dados de urgência a cada 15 minutos, bem como submissões mensais de consultas e cirurgias, o que exige um sistema capaz de lidar com frequências de atualização distintas e volumes de dados variáveis.

No que se refere à caracterização dos dados, identificaram-se problemas críticos que influenciaram o processo de desenvolvimento, tais como:

- **Dados omissos ou errados:** O histórico fornecido em formato CSV, utilizado como prova de conceito, apresenta registos com lacunas ou inconsistências que obrigam à implementação de processos de validação e tratamento antes da integração final.

- **Diversidade de formatos:** A necessidade de converter documentos XML validados por esquemas XSD para documentos JSON estruturados em MongoDB, assegurando que campos como timestamps e tipos de triagem mantenham a integridade semântica durante a transformação.

Implementação

3.1. Estruturação dos Dados no Contexto do MongoDB

Para a implementação do sistema Health Time, optou-se por uma modelação orientada a documentos utilizando o formato **JSON** (JavaScript Object Notation) do MongoDB. Esta abordagem permite uma estrutura flexível e hierárquica, ideal para representar a complexidade dos dados hospitalares.

A base de dados (HealthTime) foi organizada em quatro coleções com os dados fornecidos nos csv:

- **Hospitais:** Atua como uma coleção de metadados das instituições. Contém atributos como HospitalID, HospitalName, District e objetos de contacto (PhoneNum, Email).

- **Servicos:** Armazena a caracterização das especialidades médicas e cirúrgicas. Inclui atributos como ServiceKey, Speciality e TypeDescription (Consulta ou Cirurgia), servindo de referência para as comparações oncológicas.

- **TemposEsperaEmergencia:** Regista os fluxos de urgência registados a cada 15 minutos. A sua estrutura destaca-se pelo uso de objetos embebidos no atributo Triage, onde cada cor de triagem (Red, Orange, Yellow, Green, Blue) contém

subatributos para o tempo de espera (Time) e o número de utentes (Length).

- **TemposEsperaConsultaCirurgia:** Documenta o histórico mensal de listas de espera. Estrutura atributos como AverageWaitingTime, NumberOfPeople, Year e Month, permitindo a análise de tendências de longo prazo e a identificação de discrepâncias entre tipos de serviço.

Foram também criadas mais 5 coleções na base de dados (HealthTime) para padronizar os dados dos csv , são nestas coleções onde colocamos a informação dada pela api de consumo de XML:

- **HospitaisSEC:** Contem a informação dos hospitais padronizada onde só é colocada a informação que é pertinente, cada documento contém os seguintes campos (HospitalKey, HospitalName,Description, Address, District, Contacts ,Location,NUTS).

- **ServiçosSEC:** Normaliza a informação sobre as especialidades médicas e cirúrgicas extraídas dos relatórios submetidos. Cada documento atua como um catálogo centralizado, armazenando o identificador único do serviço (servicekey ou serviceId), a designação da especialidade (Speciality) e o tipo de serviço associado. Esta coleção permite a uniformização das referências cruzadas e a validação de especialidades entre os registos de consultas e cirurgias.

- **ConsultasSEC:** Centraliza os dados mensais de consultas externas, estruturados conforme o vocabulário definido em Consultations.xsd. A estrutura do documento no MongoDB "achata" o cabeçalho do XML, associando diretamente o InstitutionId e o Period (Mês/Ano) aos dados operacionais. Os registos incluem a segmentação por TargetPopulation (Adulto/Criança) , um objeto WaitingListCounts (com contagens para Geral, Oncológica, Não-Oncológica) e um objeto AverageResponseTimes que detalha os tempos de resposta por nível de prioridade (Normal, Prioritário, Muito Prioritário) , facilitando agregações por especialidade

- **CirurgiasSEC:** Armazena os dados mensais da atividade cirúrgica, refletindo a estrutura do Sugeri.xsd. Cada documento integra o InstitutionId e o ReferencePeriod com os detalhes da intervenção. Destaca-se o armazenamento da prioridade normalizada (priority) e das métricas de lista de espera (WaitingListCounts), bem como o tempo médio de espera (AverageWaitTimeDays) armazenado como decimal para maior precisão. Esta estrutura permite a comparação direta entre o fluxo de entrada de doentes (Consultas) e a resolução cirúrgica.

- **UrgenciasSEC:** Destina-se ao armazenamento de alta frequência (a cada 15 minutos) dos registos de urgência, baseados no Urgency.xsd. Preserva o Timestamp exato da submissão, o hospitalId e a tipologia (emergencyType). A estrutura de dados utiliza objetos aninhados para representar as escalas de triagem tanto para utentes em espera (waiting) como em observação (ObservingPatients). Cada nível da triagem (NonUrgent, LessUrgent, Urgent, VeryUrgent) é mapeado num campo específico, permitindo consultas temporais granulares sobre a afluência e o estado de abertura (OpenState) do serviço.

Esta estrutura permite que o sistema lide com dados provenientes de diferentes fontes (XML e CSV), garantindo que dados complexos sejam armazenados de forma acessível para o processamento em tempo real.

3.2. Descrição do vocabulário XML

O vocabulário XML desenvolvido tem como objetivo normalizar a comunicação de dados entre as unidades hospitalares e o sistema central "Health Time". De forma a garantir a consistência, clareza e validação dos dados , foram definidos três esquemas XSD (*XML Schema Definition*) distintos, correspondentes aos três processos de monitorização exigidos: Urgência, Consultas e Cirurgias.

Abaixo descreve-se a estrutura de cada documento, bem como as opções de tipagem e restrições aplicadas para assegurar a integridade da informação.

3.2.1 Documento das Urgências

O documento de urgência foi desenhado para suportar a submissão de dados com uma periodicidade de 15 minutos. O elemento raiz é o UrgencyRegister, que encapsula toda a informação numa estrutura hierárquica dividida em Header e Data.

- **Cabeçalho (Header):** Contém os metadados essenciais para a identificação da origem e do momento da recolha. Inclui o hospitalId (inteiro não negativo), o HospitalName e um Timestamp do tipo xs:dateTime , garantindo a precisão temporal exigida para a análise de picos de afluência.

- **Dados (Data):** Agrega a informação operacional do serviço de urgência:
- **Tipologia e Estado:** O tipo de urgência é validado através de uma *enumeration* (UrgencyTypologyType), restringindo os valores a opções predefinidas como "Urgência Geral", "Urgência Pediatrica", entre outras. O estado do serviço é controlado pelo tipo OpenStateType, aceitando apenas "Aberta" ou "Fechada".

3.2.2 Documento das consultas

Este documento destina-se ao reporte mensal da atividade de consultas externas. O elemento raiz ConsultationsRegister estrutura a informação da seguinte forma:

- **Cabeçalho (Header):** Identifica a instituição e define o período de reporte. O período é composto pelos elementos Month e Year. Foi aplicada uma restrição (xs:minInclusive value="1" e xs:maxInclusive value="12") ao elemento mês para impedir datas inválidas.
- **Dados (Data):** Contém uma lista de elementos Consultation (ilimitada via maxOccurs="unbounded"), onde cada registo detalha uma especialidade médica.
- **Segmentação:** Inclui o elemento TargetPopulation, restringido via *enumeration* aos valores "Adulto" e "Criança".
- **Tempos e Listas:** As contagens da lista de espera (WaitingListCounts) são separadas em Geral, Não-Oncológica e Oncológica. Os tempos médios de resposta (AverageResponseTimes) são tipados como NonNegativeDecimal (decimais não negativos) para maior precisão, abrangendo os níveis de prioridade "Normal", "Prioritário" e "Muito Prioritário".

3.2.3 Documentos das cirurgias

O vocabulário para cirurgias foca-se na monitorização das listas de espera cirúrgicas e tempos médios. O esquema define o elemento raiz SurgeryReport.

- **Cabeçalho (Header):** Utiliza o tipo xs:gYearMonth no elemento ReferencePeriod (ex: 2025-11), simplificando a validação do formato ano-mês exigido para o reporte mensal.

- **Dados Cirúrgicos (SurgicalData):** Composto por uma lista de SurgeryEntry.

- **Prioridade Normalizada:** O nível de prioridade é controlado pelo priorityType, uma enumeration que mapeia códigos a descrições (ex: "2 - Doença oncológica"), garantindo que os dados de entrada seguem a classificação clínica esperada.

- **Reutilização de Tipos:** O esquema define o tipo complexo WaitingListCountsType para as contagens de lista de espera (Geral, Não-Oncológica, Oncológica), promovendo a modularidade do esquema. O tempo médio de espera (AverageWaitTimeDays) utiliza a restrição NonNegativeDecimal, assegurando que não são inseridos valores de tempo negativos.

3.3.1 Pipeline de Integração e Validação XML

A integração de dados na plataforma é realizada através de um pipeline de validação e processamento de documentos XML introduzida e desenvolvida na api de consumo, garantindo a consistência estrutural e lógica da informação antes de armazenar na base de dados.

A pipeline inicia-se quando um hospital ou sistema externo submete um documento XML para a API, através de um endpoint REST protegido por autenticação. O documento é recebido com o tipo de conteúdo application/xml e processado por uma cadeia de middlewares.

Após a receção do pedido HTTP, o corpo do XML é inicialmente convertido para um objeto JavaScript através de um parser XML. Esta conversão permite o acesso estruturado aos elementos do documento, preservando a hierarquia definida no esquema XML.

Caso o XML não seja sintaticamente válido (por exemplo, erros de formatação, tags mal fechadas ou codificação inválida), o pedido é imediatamente rejeitado, sendo devolvida uma resposta de erro ao consumidor da API.

De seguida, o documento XML é validado contra um **esquema XSD específico**, correspondente ao tipo de dados submetido (consultas, urgências ou cirurgias). Esta validação assegura que:

- O elemento raiz é o esperado;

- A estrutura do documento respeita a ordem e hierarquia definidas;
- Os tipos de dados são corretos (inteiros, datas, decimais, enumerações);
- Os valores pertencem aos domínios permitidos (ex.: tipos de urgência, prioridades clínicas).

Se a validação falhar, o pipeline interrompe o processamento e devolve uma mensagem de erro detalhada, indicando a causa da rejeição (ex.: elemento inesperado, valor inválido ou ausência de campos obrigatórios).

Após todas as validações, os dados são normalizados e armazenados em coleções específicas da base de dados MongoDB, mantendo uma separação entre dados brutos importados e coleções agregadas.

Em qualquer ponto do pipeline, caso ocorra um erro, o sistema devolve uma resposta normalizada em formato JSON, com um código HTTP apropriado e uma mensagem explicativa, garantindo rastreabilidade e facilidade de diagnóstico por parte do consumidor da API.

3.3.2. Pipeline de Integração e Validação CSV

Os dados históricos disponibilizados em formato CSV são integrados no sistema através de pipelines de agregação do MongoDB, concebidas para transformar, validar e normalizar a informação antes da sua persistência nas coleções finais.

Este processo foi realizado para garantir a consistência dos dados, evitar redundância e assegurar que apenas informação relevante e validada é utilizada nas análises subsequentes.

3.3.2.1 Integração dos Dados de Urgência

Os dados relativos aos tempos de espera nas urgências são processados através de uma pipeline que transforma a estrutura original do CSV num modelo normalizado e orientado à análise.

Durante este processo:

- É estabelecida a associação entre cada registo e a instituição hospitalar correspondente, através do identificador do hospital.

- A tipologia da urgência é extraída e normalizada.
- O instante temporal do registo é convertido para o tipo `Date`, garantindo a correta realização de consultas temporais.
- O número de utentes em espera é organizado por categorias de triagem (não urgente, pouco urgente, urgente e muito urgente).

A pipeline termina com a operação `$merge`, que insere os documentos na coleção **`UrgenciasSEC`**, assegurando a consistência do processo e prevenindo a duplicação de dados aquando da reexecução.

3.3.2.2 Integração dos Dados de Consultas

Os dados de tempos de espera para consultas são integrados através de uma pipeline que combina informação proveniente de diferentes fontes, nomeadamente os ficheiros CSV e a coleção de serviços clínicos.

Neste processo:

- É realizada uma junção (`$lookup`) com a coleção de serviços, permitindo validar a tipologia do serviço associada a cada registo.
- Apenas os serviços classificados como consultas são integrados, sendo automaticamente excluídos registos não relevantes.
- São projetados os campos essenciais para a análise, incluindo a especialidade, a prioridade, o período temporal e os indicadores de tempo médio de espera e número de utentes.
- A estrutura final dos documentos é junta de acordo com o modelo definido para a coleção **`ConsultasSEC`**.

A utilização da operação `$merge` garante que os dados são inseridos de forma controlada, mantendo a consistência do sistema mesmo em cenários de reprocessamento dos ficheiros de origem.

3.3.2.3 Integração dos Dados de Cirurgias

De forma semelhante às consultas, os dados relativos às cirurgias são integrados através de uma pipeline específica, adaptada às características deste tipo de serviço.

Esta pipeline:

- Valida a tipologia do serviço através de uma junção com a coleção de serviços clínicos.
- Filtra exclusivamente os registos associados a cirurgias programadas.
- Normaliza os campos relativos à especialidade, prioridade clínica, período temporal e tempos médios de espera.
- Prepara os dados para análises comparativas entre listas de espera gerais e oncológicas.

Os documentos resultantes são integrados na coleção **`CirurgiasSEC`** através de uma operação `$merge`, assegurando a integridade e a não duplicação da informação.

Para além das pipelines principais, foram igualmente integrados dados de referência relativos a hospitais e serviços. Estas coleções são utilizadas como suporte às restantes pipelines, permitindo validar associações institucionais e tipologias de serviços, sem introduzir redundância no modelo de dados.

O MongoDB Aggregation Framework foi utilizado para integrar e preparar os dados históricos, garantindo um modelo consistente e adequado à realização das consultas analíticas solicitadas.

3.4.1 Descrição da API de consumo de documentos XML

A API desenvolvida tem como objetivo permitir a submissão, validação e armazenamento de documentos XML provenientes de entidades hospitalares, funcionando como um ponto central de integração de dados clínicos.

3.4.1.1 Estrutura da API

A API segue uma arquitetura RESTful, implementada em Node.js com o framework Express.js, e encontra-se organizada por domínios funcionais, nomeadamente:

- Consultas externas;
- Urgências hospitalares;
- Cirurgias.

Cada domínio possui um conjunto de rotas próprias, responsáveis pela receção dos documentos XML correspondentes.

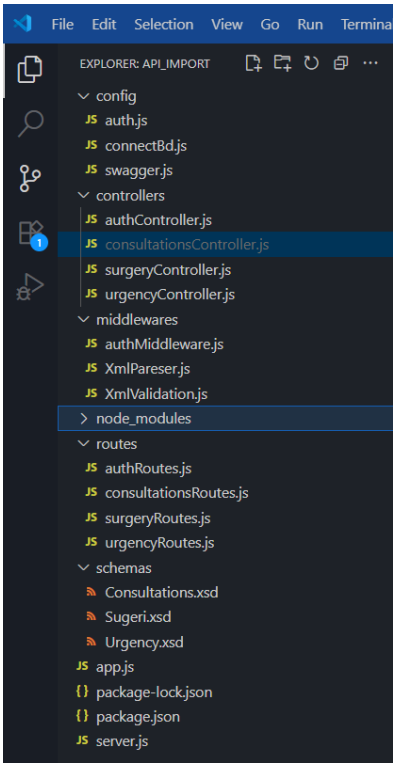


Figura 1. Estretura da API

Como podemos analisar na figura 1 a estrutura do projeto encontra-se organizada nas seguintes pastas principais:

- **routes/** – definição das rotas HTTP por domínio funcional;
- **controllers/** – implementação da lógica de negócio associada a cada tipo de documento;
- **middlewares/** – componentes reutilizáveis para autenticação, validação e pré-processamento dos pedidos;
- **schemas/** – esquemas XSD utilizados na validação dos documentos XML;
- **config/** – configuração da ligação à base de dados e parâmetros globais da aplicação;

3.4.1.2 Gestão de Erros e Respostas

A API devolve respostas claras e consistentes, utilizando códigos HTTP padrão:

- 201 Created para documentos aceites;
- 400 Bad Request para erros de validação;
- 401 Unauthorized para falhas de autenticação;
- 500 Internal Server Error para erros inesperados.

3.4.1.3 Endpoints de Submissão

A submissão dos documentos é realizada através de pedidos HTTP do tipo POST, com endpoints dedicados a cada domínio, nomeadamente:

- `/api/consultations/submit`
- `/api/urgency/submit`
- `/api/surgeries/submit`

Estes endpoints aceitam exclusivamente documentos XML no corpo do pedido e estão protegidos por um mecanismo de autenticação baseado em JSON Web Tokens (JWT), garantindo que apenas entidades devidamente autorizadas podem submeter informação para processamento.

3.4.1.4 Processo de pedidos

Cada pedido submetido à API percorre uma cadeia bem definida de middlewares, assegurando validação e segurança antes do processamento final:

1. **Autenticação**
   O middleware de autenticação valida o token JWT presente no cabeçalho Authorization. Caso o token seja inválido ou inexistente, o pedido é imediatamente rejeitado.
2. **Validação XML**
   O documento XML recebido é validado contra o respetivo esquema XSD,

garantindo a conformidade estrutural e semântica dos dados. Apenas documentos válidos prosseguem para a fase seguinte.

3. **Controller de domínio**
   O controlador correspondente ao domínio processa o XML convertido para JSON, aplica regras de negócio (por exemplo, verificação de serviços existentes ou cálculo de totais) e prepara os dados para persistência.

Esta separação de responsabilidades contribui para um fluxo de processamento claro, reduzindo erros e facilitando a reutilização dos componentes.

Os documentos XML válidos são convertidos para o formato JSON e armazenados em coleções específicas do MongoDB.

3.4.2. Descrição da API de Consulta

A API de consulta foi desenvolvida sobre a plataforma **Node.js** utilizando a *framework* **Express**, seguindo os princípios da arquitetura **REST**. A comunicação é efetuada através do método HTTP **GET**, uma vez que o objetivo é a leitura e extração analítica de dados sem alteração do estado do servidor.

A arquitetura da solução baseia-se na modularização das rotas através do `express.Router()`. A ligação à base de dados é gerida por um módulo de configuração central (`getDb`), garantindo que a conexão ao MongoDB é reutilizada eficientemente em cada pedido.

Todas as respostas da API são devolvidas em formato **JSON** (*JavaScript Object Notation*), permitindo uma integração com o ***dashboard*** (MongoCharts). Foi implementado um tratamento de erros global em cada rota (`try...catch`), onde falhas no processamento ou na base de dados devolvem um código de estado HTTP **500** e uma mensagem de erro.

As rotas foram desenhadas para serem dinâmicas, aceitando parâmetros de filtragem através de *Query Strings* (ex: `?ano=2024&hospital=HospX`). O processamento destes parâmetros inclui:

- **Conversão de Tipos:** A API converte automaticamente as *strings* recebidas para os tipos de dados corretos do MongoDB, como `Number()` para IDs e anos, e `new Date()` para filtragem de intervalos (`timestamp`).

- **Expressões:** Para pesquisas textuais (ex: tipos de urgência ou especialidades), utilizam-se expressões (`$regex`) que permitem encontrar correspondências parciais.

A lógica não é realizada no código JavaScript, mas sim transferida ao motor do MongoDB através de **Pipelines de Agregação**. A API utiliza operadores avançados como:

- `$lookup`: Para realizar junções (*joins*) entre coleções, como associar dados da coleção `UrgenciasSEC` com os detalhes geográficos da coleção `HospitaisSEC`.

- `$group` e `$project`: Para agrupar dados por categorias (ex: prioridade de triagem, região NUTS) e formatar a saída.

Como exemplo, destaca-se a rota `/discrepancia-servicos`. Este *endpoint* tem como objetivo comparar a eficiência de resposta entre dois serviços distintos para o mesmo hospital e especialidade.

- **Entrada:** A rota recebe os parâmetros `hospital`, `especialidade` e ano.

- **Processamento:** Ao contrário de outras rotas que consultam uma única coleção, este algoritmo executa duas operações de agregação distintas e sequenciais:
    - Consulta a coleção `ConsultasSEC` para calcular o tempo médio de espera (`averageWaitingTime`) para os critérios definidos.

    - Consulta a coleção `CirurgiasSEC` aplicando os mesmos filtros.

Output:


```
{
  "mediaConsulta": 54.333333333333336,
  "mediaCirurgia": 54.333333333333336
}
```


Para além do exemplo descrito, a API disponibiliza um conjunto de *endpoints* que respondem diretamente aos requisitos analíticos do projeto:

1. Cálculo da média de utentes em espera para 1ª observação médica, filtrada por intervalos de tempo dinâmicos e discriminada pelas categorias de triagem (não urgente, pouco urgente, urgente, muito urgente).

2. Análise da distribuição percentual de utentes por categoria de triagem em relação ao total, com segmentação automática pelo período do dia (manhã, tarde, noite) num determinado hospital.

3. Agregação dos tempos médios de espera em urgências pediátricas, agrupados por região geográfica, permitindo a análise de desempenho territorial.

4. Análise diferencial dos tempos médios de resposta em consultas de especialidade, distinguindo entre prioridades oncológicas e não-oncológicas.

5. Comparação dos tempos médios de espera para cirurgia programada por especialidade, confrontando a lista geral com a lista oncológica.

6. Cruzamento de dados para avaliar o desfasamento entre o tempo de resposta para consulta e o tempo de espera para cirurgia, detalhado por hospital e especialidade.

7. Identificação dos 10 hospitais com menores tempos médios de espera em urgências pediátricas, enriquecendo a resposta com dados institucionais (nome, região e contactos).

8. Algoritmo de monitorização da afluência às urgências gerais ao longo do dia, com agregação de dados em intervalos de 15 minutos para identificação precisa dos picos de maior afluência (Top 3).

3.5 Implementação das consultas

Abaixo descreve-se a lógica de construção das pipelines para cada endpoint, identificando os operadores e o motivo da sua utilização.

3.5.1. Média de Utentes em Espera (`/media-espera`)

- **`$match`**: Utilizado para filtrar os documentos pelo intervalo de datas (`timestamp`). Foi essencial converter as strings de input com `new Date()` para garantir que a comparação temporal funcionava corretamente.

- **`$group`**: Agrupou-se pelo campo `emergencyType` para separar as diferentes tipologias de urgência.

- **`$avg`**: Aplicado dentro do agrupamento para calcular a média aritmética dos campos aninhados em `waiting` (nonUrgent, urgent, lessUrgent, veryUrgent.).

- **`$project` e `$round`**: Utilizados no estágio final para limpar o output (removendo o `_id`) e arredondar os valores decimais a 2 casas, facilitando a leitura do JSON.

3.5.2 Percentagem por Triagem e Período do Dia (`/percentagem-triagem`)

- **`$addFields`**: Criou-se o campo `hora` (usando o operador **`$hour`**) e o campo `periodoDia`.

- **`$switch`**: Fundamental para implementar a lógica de "Manhã/Tarde/Noite". Atua como um *case-switch* que avalia a hora extraída e atribui a categoria correspondente.

- **`$objectToArray`**: Como o campo `waiting` era um objeto (e não um array), usou-se este operador para o transformar numa lista de chave-valor (k, v), permitindo a sua manipulação posterior.
- **`$unwind`**: "Desconstruiu" o array gerado anteriormente para permitir agrupar e somar os totais individualmente por categoria de triagem.

- **`$divide` e `$multiply`**: Usados no estágio final para calcular a percentagem matemática (Valor / Total * 100).

3.5.3 Média de Pediatria por Região (`/media-pediatria-regiao`)

- **`$match` com `$regex`**: Filtrou-se a tipologia usando expressões regulares para obter variações de "Pediatria" ou "Pediátrica" sem distinção de maiúsculas/minúsculas.

- **`$lookup`**: Operador de junção (*join*) essencial para cruzar a coleção de factos (`UrgenciasSEC`) com a coleção de dimensões (`HospitaisSEC`), utilizando o `hospitalId`. A sua utilização foi obrigatória dado que a coleção de urgências armazena apenas a referência numérica (`hospitalId`), desconhecendo a localização do hospital; este operador permitiu aceder à informação das regiões, que reside exclusivamente na coleção de hospitais, viabilizando assim o agrupamento e cálculo dos tempos médios por região geográfica.

- **`$unwind`**: Necessário logo após o lookup, pois este devolve um array. O unwind transforma-o num objeto simples para se poder aceder a `hospital.NUTS`.

- **`$add` dentro de `$avg`**: Somaram-se primeiro todas as categorias de prioridade (`waiting.urgent` + `nonUrgent`...) para obter o total de utentes antes de calcular a média por região.

3.5.4 Comparativo Oncologia vs. Não-Oncologia (`/comparativo-oncologia`)

- **`$cond` (Condicional)**: Utilizado dentro do estágio `$group` para criar buckets de agrupação dinâmicos.

- **`$regexMatch`**: Integrado na condição para analisar a string de prioridade. Se a descrição contiver "oncol", o registo é agrupado como "Oncologia", caso contrário, "NaoOncologia". Isto evitou a necessidade de fazer duas queries separadas.

3.5.5 Comparativo Cirurgia (`/comparativo-cirurgia`)

- **`$match`**: Filtragem rigorosa por Hospital, Ano e Especialidade para garantir a comparabilidade dos dados.

- **`$group`**: Agrupamento composto por `especialidade` e `lista` (prioridade) para obter a média direta.

3.5.6. Discrepância Consulta vs. Cirurgia (`/discrepancia-servicos`)

- Optou-se por executar duas agregações leves e paralelas (`ConsultasSEC` e `CirurgiasSEC`) com `$match` e `$group` idênticos. Como as coleções são semanticamente diferentes e não têm uma chave de ligação direta 1:1, a unificação dos dados foi feita de forma mais eficiente ao nível do código (Node.js) do que tentar forçar um `$lookup` complexo.

3.5.7 Top 10 Hospitais Pediátricos (`/top10-hospitais`)

- **`$first`**: Utilizado dentro do `$group` para manter os dados estáticos do hospital (Nome, Contacto, Região) após o agrupamento, evitando que se perdessem ou duplicassem.

- **`$sort`**: Ordenação ascendente (1) pelo campo da média calculada, para colocar os hospitais com menores tempos de espera no topo.

- **`$limit`**: Restringiu o output apenas aos primeiros 10 documentos, otimizando o tamanho da resposta da API.

3.5.8 Evolução Temporal e Picos (`/evolucao-temporal-geral`)

- **(`$mod`, `$subtract`)**: Esta foi a operação mais complexa. Para criar intervalos de 15 minutos:
  - Extraiu-se o minuto atual (`$minute`).
  - Calculou-se o resto da divisão por 15 (`$mod`).
  - Subtraiu-se esse resto ao minuto original (`$subtract`).

- **`$add`**: Somaram-se as 4 categorias de urgência (verde, amarelo, laranja, vermelho) num campo temporário `afluencia` antes de agrupar, para ter o volume total de pessoas naquele intervalo.

- **`$sort`**: Ordenação primeiro por Hora e depois por Intervalo, garantindo que o gráfico de linha é desenhado cronologicamente.

3.6. Descrição da Dashboard em MongoCharts

Para a componente de visualização de dados, optou-se pela utilização do **MongoDB Charts**. Esta ferramenta permitiu criar visualizações em tempo real diretamente sobre documentos JSON.

A construção do *dashboard* foi orientada pela necessidade de **monitorizar indicadores críticos** de funcionamento dos hospitais .O objetivo foi transformar os dados brutos das coleções **SEC** em *insights* funcionais para a gestão hospitalar. Foram desenvolvidos quatro *dashboards* distintos, focados em quatro pontos fundamentais neste trabalho: **Eficiência** (comparação de tempos), **Procura** (volume por especialidade), **Evolução**(padrões de afluência) e **Geografia** (distribuição territorial).

Abaixo descreve-se cada um dos *dashboards* desenvolvidos:

1. Evolução Temporal de Afluência (Urgências)

- **Visualização:** Gráfico de Linha Discreto (*Discrete Line*).
- **Fonte de Dados:** Coleção `UrgenciasSEC`.
- **Descrição:** Eixo X definido pelo `timestamp` (com *binning* automático ou horário) e Eixo Y a representar a média ou contagem de utentes não urgentes (`waiting.nonUrgent`).
- **Objetivo:** Este gráfico é essencial para a gestão de turnos, pois revela os padrões de afluência ao longo do tempo. Permite identificar picos de procura, ajudando a prever a necessidade de reforço de triagem.

2. Tempo Médio de Espera (Consultas)

- **Visualização:** Gráfico de Barras (*Grouped Column*).
- **Fonte de Dados:** Coleção `ConsultasSEC`.
- **Descrição:** Este gráfico agrega os dados por Hospital (Eixo X) e calcula a média (mean) do campo `averageWaitingTime` (Eixo Y). Os dados são ordenados de forma decrescente.
- **Objetivo:** Permitir uma comparação direta de eficiência entre os diferentes hospitais. Identifica rapidamente quais os hospitais com maiores dificuldades de resposta (barras mais altas) e quais os que cumprem melhor os prazos.

3. Rede Nacional de Hospitais (Distribuição Geográfica)

- **Visualização:** Mapa de Dispersão (*Geo Scatter*).
- **Fonte de Dados:** Coleção `HospitaisSEC`.
- **Descrição:** Utiliza as coordenadas geoespaciais armazenadas no objeto `Location` (`Latitude` e `Longitude`) para posicionar cada unidade no mapa. As cores dos pontos são determinadas pela região `NUTS`, permitindo uma segmentação visual por território.
- **Objetivo:** Análise de cobertura territorial. O mapa oferece uma visão geral da distribuição dos recursos de saúde, permitindo identificar assimetrias regionais e visualizar a densidade da rede hospitalar.

4. Volume de Procura por Especialidade Cirúrgica

- **Visualização:** Nuvem de Palavras (*Word Cloud*).
- **Fonte de Dados:** Coleção `CirurgiasSEC`.
- **Descrição:** O gráfico utiliza o campo `speciality` para o texto e a soma (`SUM`) do campo `numPatients` para definir o tamanho de cada palavra. Foi aplicado um filtro de limite de resultados para garantir clareza nos dados.
- **Objetivo:** Destacar visualmente as áreas médicas sob maior pressão. A nuvem de palavras permite perceber instantaneamente quais as especialidades (ex: Oftalmologia, Ortopedia) que acumulam maior número de doentes em lista de espera, sugerindo onde deve ser feito o reforço de equipas médicas.

Acesso à Dashboard: A dashboard encontra-se no MongoAtlas através deste link:

https://charts.mongodb.com/charts-trabalho-de-pei-jgtwusp/public/dashboards/695460e8-5d0a-4e19-8591-4a561bf4a864

4. Apreciação Crítica

4.1. Avaliação da Abordagem seguindo os Requisitos

A solução desenvolvida foi alinhada com os requisitos definidos no enunciado do projeto, nomeadamente no que respeita à uniformização da informação hospitalar, à integração de dados provenientes de múltiplas fontes e à realização de análises comparativas sobre tempos de espera. A utilização de vocabulários XML validados por esquemas XSD permitiu garantir a consistência estrutural e semântica dos dados submetidos pelas instituições.

Relativamente ao processamento e armazenamento da informação, a adoção do MongoDB e do seu Aggregation Framework permitiu cumprir os requisitos analíticos estabelecidos, assegurando a transformação, validação e agregação eficiente dos dados históricos em CSV e dos dados submetidos via XML. A implementação da REST API possibilitou o acesso dinâmico à informação, conforme solicitado, suportando filtros temporais, institucionais e clínicos.

4.2. Pontos Fortes e Limitações

Como principais pontos fortes destaca-se a flexibilidade da modelação orientada a documentos, que facilitou a representação de dados complexos e heterogéneos. A centralização da lógica de processamento nas pipelines de agregação contribuiu para uma solução eficiente e escalável, reduzindo a complexidade da camada aplicacional. Adicionalmente, a utilização de dashboards permitiu uma visualização clara e imediata dos indicadores relevantes.

Como limitações, salienta-se a dependência da qualidade dos dados de origem, em particular dos ficheiros CSV históricos, que apresentavam lacunas e inconsistências. Embora estas tenham sido mitigadas através de processos de validação, o sistema poderia beneficiar de mecanismos adicionais de controlo de qualidade e auditoria. Como melhoria futura, seria relevante automatizar a ingestão dos dados e otimizar algumas consultas para cenários de maior volume de informação.

4.3. Impacto do Trabalho Desenvolvido

O vocabulário XML desenvolvido demonstrou ser aplicável ao domínio da monitorização hospitalar, permitindo uma comunicação estruturada e uniforme entre diferentes instituições. A sua utilização contribui para a interoperabilidade dos sistemas e para a fiabilidade da informação analisada.

O trabalho desenvolvido apresenta potencial de evolução, podendo ser estendido para incluir novos indicadores clínicos, integração com outros sistemas de saúde ou mecanismos de análise preditiva. Estas extensões reforçariam o valor da solução enquanto ferramenta de apoio à decisão.

5. Conclusão

5.1. Resumo dos Resultados Obtidos

O projeto resultou na implementação de um sistema funcional para a monitorização dos tempos de espera hospitalares, capaz de integrar dados de diferentes formatos, validá-los e disponibilizar análises relevantes através de uma REST API e dashboards interativos. Os objetivos definidos foram cumpridos, demonstrando a aplicabilidade prática dos conceitos abordados na unidade curricular.

5.2. Considerações Finais

Em termos globais, o trabalho permitiu consolidar conhecimentos técnicos e conceptuais sobre processamento estruturado de informação, vocabulários XML e

bases de dados NoSQL. A solução apresentada é coerente, escalável e adequada ao domínio em estudo, constituindo uma base sólida para evoluções futuras e para a aplicação dos conceitos aprendidos em contextos reais.